



AI News & Skills 项目架构 — 视频口播脚本



小红书短视频（90-120 秒，9:16）

风格：快剪节奏，每段 10-15 秒。大数字 + 关键词弹出。

段 1 · 开场 Hook (0:00-0:10)

视觉：深色底 + 管道流动动画

一个人，一条自动化管道，把 AI 内容工厂跑起来了。这套系统叫 AI News & Skills——它能干什么？我跟你

说。

段 2 · 顶层架构 (0:10-0:22)

视觉：三栏布局卡片弹出

整个项目分三块——左边 AI News 自动化工厂，中间视频发布引擎，右边 AI Tour 教学网站。一拖二，全自

动。

段 3 · 核心引擎 (0:22-0:35)

视觉: 节点连线动画, 4 步流动

核心在 AI News 工厂, 4 层管道。第一层采集——163 邮箱、Bing、GitHub、ArXiv 全抓。第二层精选——Claude 翻译、去重、打分, 挑 6 到 8 条。第三层渲染——HTML 卡片到 PNG, 封面到视频, 全自动。第四层分发——小红书、B站、飞书、Vercel, 一键推。

段 4 · 两个品牌 (0:35-0:48)

视觉: 左蓝绿 vs 右暖色, 翻转对比

两种内容格式, 两个品牌配置——AI News 周报, 默认跑, 蓝绿配色, 6 到 8 条新闻加 3 个 GitHub 项目。AI Skills 深度, 手动触发, 暖色调, 白皮书风格长图文。设计 token 全外部化, 改视觉不碰代码。

段 5 · Claude Code Skills (0:48-1:00)

视觉: 紫色 Card UI 翻转

还内置了 3 个 Claude Code Skill——「生成 AI 内容工厂卡片」, 一句话触发 6 张卡片。「参考这个样式」, 截图直接提取设计 token。竞品追踪, 自动监控同类账号。

段 6 · 收尾 (1:00-1:10)

视觉: 全景架构俯瞰 → Logo

从采集到发布，从卡片到视频——一个人搭建的内容工厂，全链路跑通。这就是 AI News & Skills 的项目架构。想搭自己的？先看懂这张图。

总时长: ~80 秒

TTS 语速: 2.0x (智谱"锤锤"或 Azure"晓晓")

BGM: 低音电子 / 科技感, bgm/mixkit_132.mp3

视觉帧: 每段 1-2 帧, 架构图 + 大数字叠加



B 站长视频 (8-12 分钟, 16:9)

风格: 叙事纪录片, 章节过渡, 架构图逐层展开, 有观点句停顿。

开场 (0:00-0:45)

今天不讲新闻，讲工具本身。我想带你拆一套我自己搭的 AI 内容自动化系统。

>

它的名字叫 AI News & Skills。从一条邮件到一张小红书卡片，从一篇翻译到一条 B 站视频——全自动。

>

这套系统我已经跑了几个月，产出了几十条内容。今天把它拆开给你看——架构是什么，为什么这么设计，每一层做对了什么。

>

我们从头开始。

第 1 章 · 顶层布局 — 三个子系统 (0:45-2:00)

打开项目目录，第一眼看见三块——

>

第一块，ai-news-xiaohongshu。这是核心，AI 内容工厂。第二块，video-publish。视频发布管线，对接小红书和 B 站。第三块，website。AI Tour Education 教学网站，画布式 AI 工作流教学。

>

为什么分三块？因为内容生产和内容分发是两个节奏。工厂产出的是「内容资产」——卡片、文案、视频素材。发布层关心的是「平台适配」——尺寸、格式、上传 API。

>

中间用目录做接口。工厂把卡片 PNG 放到 xiaohongshu/日期/ 目录下。发布层去读这个目录，不管内容怎么来的。

>

这是第一个设计原则：生产和分了解耦。

第 2 章 · 核心工厂 — 4 层管道 (2:00-4:00)

重点拆核心——ai-news-xiaohongshu。

>

4 层管道。一层一层说。

>

****第一层：采集。****入口是 `fetch_ai_news.py`，一个 Python 脚本。它先连 163 邮箱 IMAP，搜索标题含 AINews 的邮件，近 7 天。拉下正文后，调用 Claude API 做中英互译。翻译不是终点，是起点——翻译完的内容作为后续精选的「种子语料」。

>

同时跑全网抓取。`web_scraper.py`，接 5 个信息源——Bing 搜索、Hacker News Algolia API、Reddit JSON、Twitter Nitter 镜像、ArXiv Atom XML。`github_trending.py` 调 GitHub Search API，拉 24 小时 star 增速。

>

这层的设计原则是：****邮件是锚点，但不止于邮件。****邮件告诉你「发生了什么」，全网抓取补足「还发生了什么」。

>

****第二层：精选。****`item_matcher.py` 接管。把所有来源——邮件翻译、网页抓取、GitHub——一起扔给 Claude。Claude 做三件事：去重、打分、精选出 6 到 8 条。然后再做一个质量预判——每条评分，低分的重写。最后生成多平台文案——小红书、B 站、Twitter，各写一套。

>

这里有个关键设计：****人机回环确认。****精选结果出来之后，终端打印出来，等人按 y 或 n。为什么？因为 Claude 的判断力，在「这条值不值得发」这件事上，还是不如你。自动化不是取代判断，是帮你省掉 90% 的体力活，保留最后 10% 的决策权。

>

****第三层：渲染。****`xhs_publish/` 目录接管。`render_combined.py` 生成卡片 HTML——1080 乘 1440 像素，2 倍缩放。Playwright 无头 Chrome 截图，输出 PNG。同时生成两种封面——小红书 3 比 4 竖屏，B 站 16 比 9 横屏。

>

视频层更重——`render_video.py` 调 FFmpeg 合成。每张卡片 5 秒，封面 3 秒，中间 0.4 秒转场。Azure TTS 语音合成，智谱 TTS 做 fallback。`bgm` 目录里扔几首 mp3，自动混入，音量 0.08。

>

渲染层最容易被低估。但内容是「看起来怎么样」决定的。所以视觉 token 不在代码里，在 brands/ 目录的两个 JSON 文件里。所有颜色、字号、圆角、阴影，全是外部化配置。想换风格？改 JSON，不碰一行代码。

>

****第四层：分发。**** 最后一步。飞书 Webhook 推卡片消息。HTML 复制到 Vercel public 目录。小红书自动发布用 Playwright 模拟浏览器登录创作者中心，上传图片和文案。

>

四层走完，从一封邮件到全平台发布，全程只需要点一次 y。

第 3 章 · 两个品牌 — 一套系统，两种输出 (4:00-5:30)

同一个管道，出两种内容。

>

****AI News 周报——默认模式，每周日跑。**** 蓝绿配色，Noto Sans SC 字体，900 字重。3 层 info-card：发生了什么、深入了解、为什么值得关注。加一个 Insight Box 金句收尾。保持 6 到 8 条新闻加 3 个 GitHub 项目。文案走「专业但轻松，小白友好，有深度但不装」的路线。

>

****AI Skills 深度——手动触发，不定期。**** 暖色调，Inter 字体，白皮书风格。4 到 5 张长图文，layout 是 steps 加 insight-box。文案走教学引导，有步骤有案例，不卖焦虑。

>

为什么需要两个品牌？因为内容和节奏不一样。周报解决的是「信息密度」——一周过去了，你该知道这 6 件事。深度解决的是「理解密度」——一个话题，你得真正搞懂。

>

品牌配置的存在，让同一个管道可以同时跑两种内容而不打架。视觉 token、字体、卡片结构、社交媒体标签

——全部在 JSON 里隔离。这是第二个设计原则：**内容结构不变，视觉表现可插拔。**

第 4 章 · Claude Code Skills — 把工具变成一行命令 (5:30-7:00)

第三个设计亮点——Claude Code Skills。

>

在 .claude/skills/ 目录下，3 个 Skill 文件。

>

****第一个：xhs-content-engine。****「生成 AI 内容工厂卡片」——一行话。触发后自动生成 6 张卡片：封面、从手动到自动化、引擎一深度拆解、引擎二深度拆解、对比表加共享基础设施、安装指南。所有内容自动填入。紫色主题，1080 乘 1440，2x 截图。

>

****第二个：style-reference。****这可能是最妙的一个。看到一张好的小红书封面，截图，说「参考这个样式」。Claude 自动提取设计 token——主色、强调色、字体、间距、圆角、阴影。生成 HTML 参考页，存到 design-refs 目录。下次生成卡片时可以引用这些 token。设计知识不是记在脑子里的，是存在文件系统里的。

>

****第三个：competitive-reference。****竞品追踪。追踪 5 个同类开源项目的更新节奏、内容策略、视觉变化。优先级 Roadmap 自动排。

>

3 个 Skill 的本质是什么？是把「经验」变成「可执行的指令」。你不在这里，别人也能用这套东西跑出一样的卡片。这是第三个设计原则：****知识不依赖人，依赖系统。****

第 5 章 · 三个设计原则 — 为什么这样搭 (7:00-8:10)

第 6 章 · 三个设计原则 — 为什么这样搭 (8:30-9:30)

拆完架构，回头看，三条贯穿始终的设计原则。

>

****第一条：生产和分发放解耦。**** 工厂产出内容资产，发布层只管平台适配。你今天加了小红书自动发布，不影响卡片生成逻辑。明天想加 YouTube Shorts，只要在发布层加一个适配器。

>

****第二条：视觉表现可插拔。**** 颜色、字体、间距、布局模式——全在 JSON 配置文件里。品牌切换不需要改代码，改一个 JSON 字段就行。设计 token 和业务逻辑彻底分离。

>

****第三条：知识不依赖人，依赖系统。**** Claude Code Skills 把经验变成可执行指令。内容偏好存 preference 文件，不是记在提示词里。竞品追踪自动跑，不用手动对比。工具知道你要什么，不是你每次都告诉它。

>

这三条加在一起，一件事就很清楚了——这套系统不是「帮一个人做内容」，是「把做内容这件事，变成一套可以复制的流程」。

结语 (8:10-8:55)

我经常说一句话：自动化不是取代判断，是替你省掉 90% 的体力活，保留最后 10% 的决策权。

>

翻译不用自己翻，精选不用自己搜，卡片不用自己排，视频不用自己剪。但要发哪条、不要哪条、这期封面用什么色调——这些决定还是你来做。

>

好的自动化，边界就在这里。它把你从重复劳动里拉出来，但把你放在决策者的位置上。

>

这就是 AI News & Skills 的项目架构。从一封邮件，到全平台发布。从一个人，到一个内容工厂。

>

如果你想搭自己的自动化管道，把这张架构图存下来。不需要一模一样——理解了这三条原则，你搭出来的东西，会比我的更好。

>

感谢看完。我是程序员老崔，我们下期见。

总时长: ~9 分钟

章节: 开场 + 5 章 + 结语 = 7 段

TTS: 智谱 GLM-TTS, "锤锤", 语速 1.8x

BGM: bgm/mixkit_371.mp3, 低音科技感, 0.08 音量

视觉帧: 架构图逐层展开 + 代码片段 + 卡片成品展示